FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE

2

Sentiment Analysis

Group Name:	
Members:	

Name	Role
Arianne Beverly V. Marquez	Programmer & Tester

Section:	**AN41**

Professor:	Ms. Crisola Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE •

Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of Sentiment Analysis.
- Design and develop a simple program about Sentiment Analysis using Python.

IV. BACKGROUND INFORMATION

Sentiment Analysis can assist us in determining the mood and feelings of the general public as well as obtaining useful information about the setting. Sentiment Analysis is the process of assessing data and categorizing it according to the research's needs.

Sentiments can be used to gain a better understanding of different events and their consequences. It goes by various names, including "sentiment analysis," "opinion mining," "opinion extraction," "sentiment mining," "subjectivity analysis," "affect analysis," "emotion analysis," and "review mining," but they all serve the same objective. We might be able to figure out what people like or want by evaluating their feelings.



TextBlob

TextBlob is a python library and offers a simple API to access its methods and perform basic NLP tasks.

Textblob's sentiment function returns polarity and subjectivity as two characteristics.

The polarity ranges from [-1,1], with 1 denoting a positive statement and -1 denoting a negative one. Subjective statements usually refer to personal feelings, emotions, or judgments, whereas objective sentences refer to facts. Subjectivity is also a float with a value between 0 and 1.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

Perform the following:

1. Add the following code:

```
from textblob import TextBlob
```

2. Declare a variable that will accept a sentence:

```
sentence = input("Ënter a sentence : ")
```

3. Next initialize TextBlob

```
s = TextBlob(sentence)
```

4. Then check the polarity of the sentence by using the code below:

```
p = s.sentiment.polarity
```

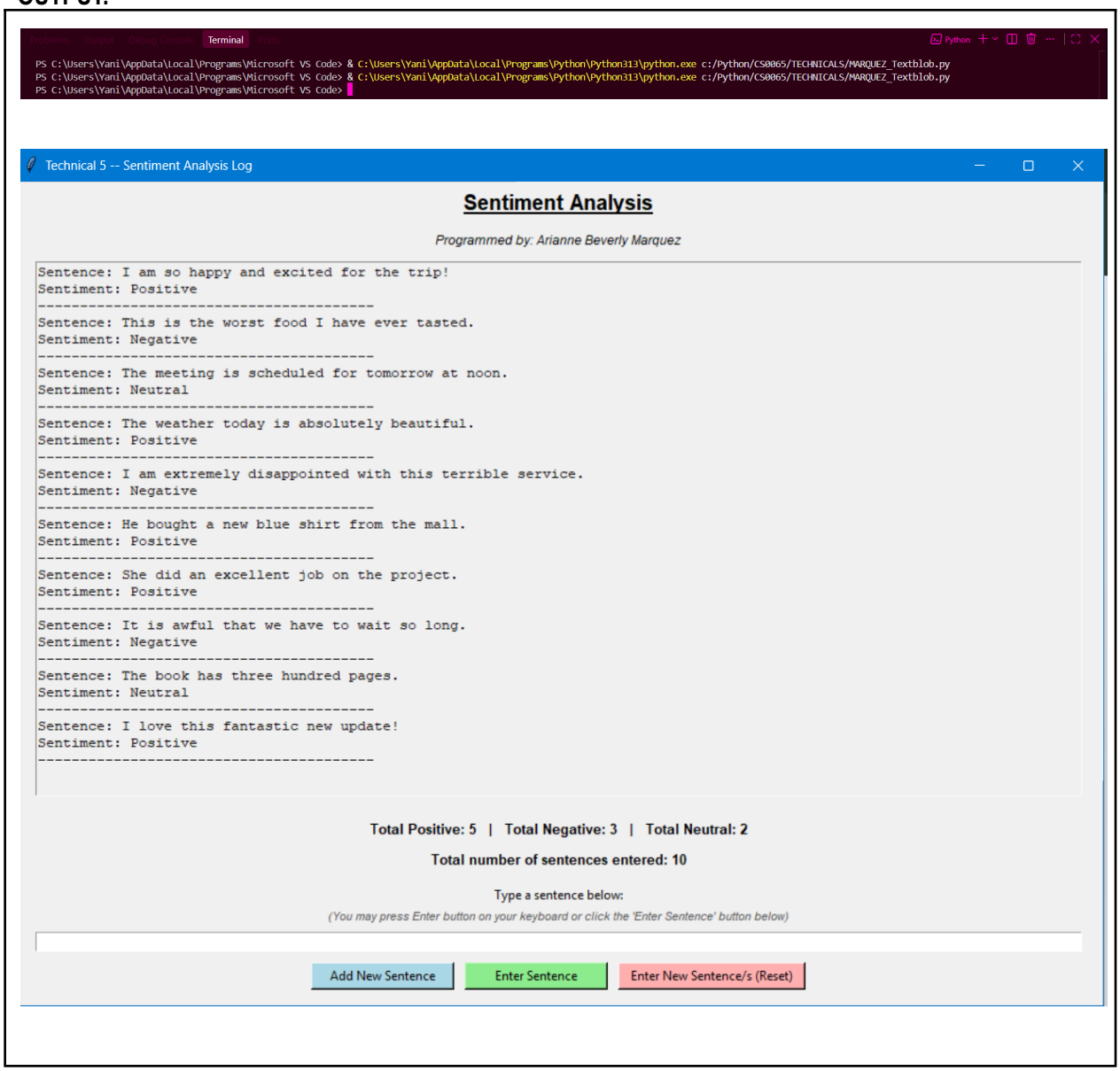
5. Make a decision based on the polarity. If the polarity of the sentence is less than zero the sentiment is negative. If the polarity is equals to zero, then the sentiment is neutral. If the polarity is greater than zero, the sentiment is positive.

```
#Negative p<0, Neutral p=0, Positive p>0
if p <0:
    print("Negative")
elif p==0:
    print("Neutral")
```

```
elif p>0 and p<=1:
    print("Positive")
```


6. Make the program perform the sentiment analysis 10 times and make a summary of how many sentences are positive, neutral or negative.

OUTPUT:



```
Sentence: I am so happy and excited for the trip!
Sentiment: Positive
----------------------------------------
Sentence: This is the worst food I have ever tasted.
Sentiment: Negative
----------------------------------------
Sentence: The meeting is scheduled for tomorrow at noon.
Sentiment: Neutral
----------------------------------------
Sentence: The weather today is absolutely beautiful.
Sentiment: Positive
----------------------------------------
Sentence: I am extremely disappointed with this terrible service.
Sentiment: Negative
----------------------------------------
Sentence: He bought a new blue shirt from the mall.
Sentiment: Positive
----------------------------------------
Sentence: She did an excellent job on the project.
Sentiment: Positive
----------------------------------------
Sentence: It is awful that we have to wait so long.
Sentiment: Negative
----------------------------------------
Sentence: The book has three hundred pages.
Sentiment: Neutral
----------------------------------------
Sentence: I love this fantastic new update!
Sentiment: Positive
----------------------------------------
```

CODE:

```
import tkinter as tk
from tkinter import messagebox
from textblob import TextBlob

# Global counters for the summary
pos_count = 0
neg_count = 0
neu_count = 0

def add_new_sentence():
    global pos_count, neg_count, neu_count

    # Get the sentence from the single-line entry
```

```
    sentence = entry.get().strip()

    if not sentence:
        messagebox.showwarning("Input Error", "Please enter a
sentence.")
        return

    # TextBlob sentiment analysis
    s = TextBlob(sentence)
    p = s.polarity

    # Make a decision based on the polarity
    if p < 0:
        sentiment = "Negative"
        neg_count += 1
    elif p == 0:
        sentiment = "Neutral"
        neu_count += 1
    elif p > 0 and p <= 1:
        sentiment = "Positive"
        pos_count += 1

    # Append the result to the history text area
    history_text.config(state=tk.NORMAL) # Enable editing
momentarily
    history_text.insert(tk.END, f"Sentence: {sentence}\nSentiment:
{sentiment}\n")
    history_text.insert(tk.END, "-"*40 + "\n")
    history_text.see(tk.END) # Auto-scroll to the bottom
    history_text.config(state=tk.DISABLED) # Disable editing

    # Update the summary labels and clear the entry box for the next
input
    update_summary()
    entry.delete(0, tk.END)

def enter_new_sentences():
    """Clears the history and resets counters to start a new batch
of sentences."""
    global pos_count, neg_count, neu_count

    # Reset counters
    pos_count = neg_count = neu_count = 0

    # Clear history text area
```

```
    history_text.config(state=tk.NORMAL)
    history_text.delete("1.0", tk.END)
    history_text.config(state=tk.DISABLED)

    # Reset summary and entry box
    update_summary()
    entry.delete(0, tk.END)

def update_summary():
    """Updates the summary tally at the bottom."""
    summary_label.config(text=f"Total Positive: {pos_count}    |
Total Negative: {neg_count}    |   Total Neutral: {neu_count}")
    total_label.config(text=f"Total number of sentences entered:
{pos_count + neg_count + neu_count}")

# Main GUI Setup
root = tk.Tk()
root.title("Technical 5 -- Sentiment Analysis Log")
root.geometry("600x550")
root.minsize(550, 450) # Prevents the window from being squeezed too
small

# Header
tk.Label(root, text="Sentiment Analysis", font=("Arial", 15, "bold",
"underline")).pack(pady=5)
tk.Label(root, text="Programmed by: Arianne Beverly Marquez",
font=("Arial", 9, "italic")).pack(pady=5)

# Bottom Frame (Input, Summary, and Buttons)
bottom_frame = tk.Frame(root)
bottom_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=20, pady=10)

# Summary Labels (Inside bottom frame)
summary_label = tk.Label(bottom_frame, text="Total Positive: 0    |
Total Negative: 0    |   Total Neutral: 0", font=("Arial", 10,
"bold"))
summary_label.pack(pady=5)

total_label = tk.Label(bottom_frame, text="Total number of sentences
entered: 0", font=("Arial", 10, "bold"))
total_label.pack(pady=2)

# Input Field and Instructions (Inside bottom frame)
tk.Label(bottom_frame, text="Type a sentence below:").pack(pady=(10,
0))
```

```
tk.Label(bottom_frame, text="(You may press Enter button on your
keyboard or click the 'Enter Sentence' button below)",
font=("Arial", 8, "italic"), fg="#555555").pack()

entry = tk.Entry(bottom_frame)
entry.pack(fill=tk.X, pady=5) # makes the entry box stretch
horizontally with the window

# Buttons Frame (Inside bottom frame)
btn_frame = tk.Frame(bottom_frame)
btn_frame.pack(pady=5)

add_btn = tk.Button(btn_frame, text="Add New Sentence",
bg="lightblue", width=18, command=add_new_sentence)
add_btn.grid(row=0, column=0, padx=5)

enter_btn = tk.Button(btn_frame, text="Enter Sentence",
bg="lightgreen", width=18, command=add_new_sentence)
enter_btn.grid(row=0, column=1, padx=5)

clear_btn = tk.Button(btn_frame, text="Enter New Sentence/s
(Reset)", bg="#ffb3b3", width=24, command=enter_new_sentences)
clear_btn.grid(row=0, column=2, padx=5)

#  Middle Frame (History Text Area)
history_text = tk.Text(root, state=tk.DISABLED, bg="#f0f0f0")
history_text.pack(fill=tk.BOTH, expand=True, padx=20, pady=5)

# Bind the Enter key
root.bind('<Return>', lambda event: add_new_sentence())

root.mainloop()
```

CODE SCREENSHOT:

Welcome MARQUEZ_Textblob.py

C: > Python > CS0065 > TECHNICALS > MARQUEZ_Textblob.py > ...

```python
import tkinter as tk
from tkinter import messagebox
from textblob import TextBlob

# Global counters for the summary
pos_count = 0
neg_count = 0
neu_count = 0

```

Welcome MARQUEZ_Textblob.py

C: > Python > CS0065 > TECHNICALS > MARQUEZ_Textblob.py > ...

```python
def add_new_sentence():
    global pos_count, neg_count, neu_count

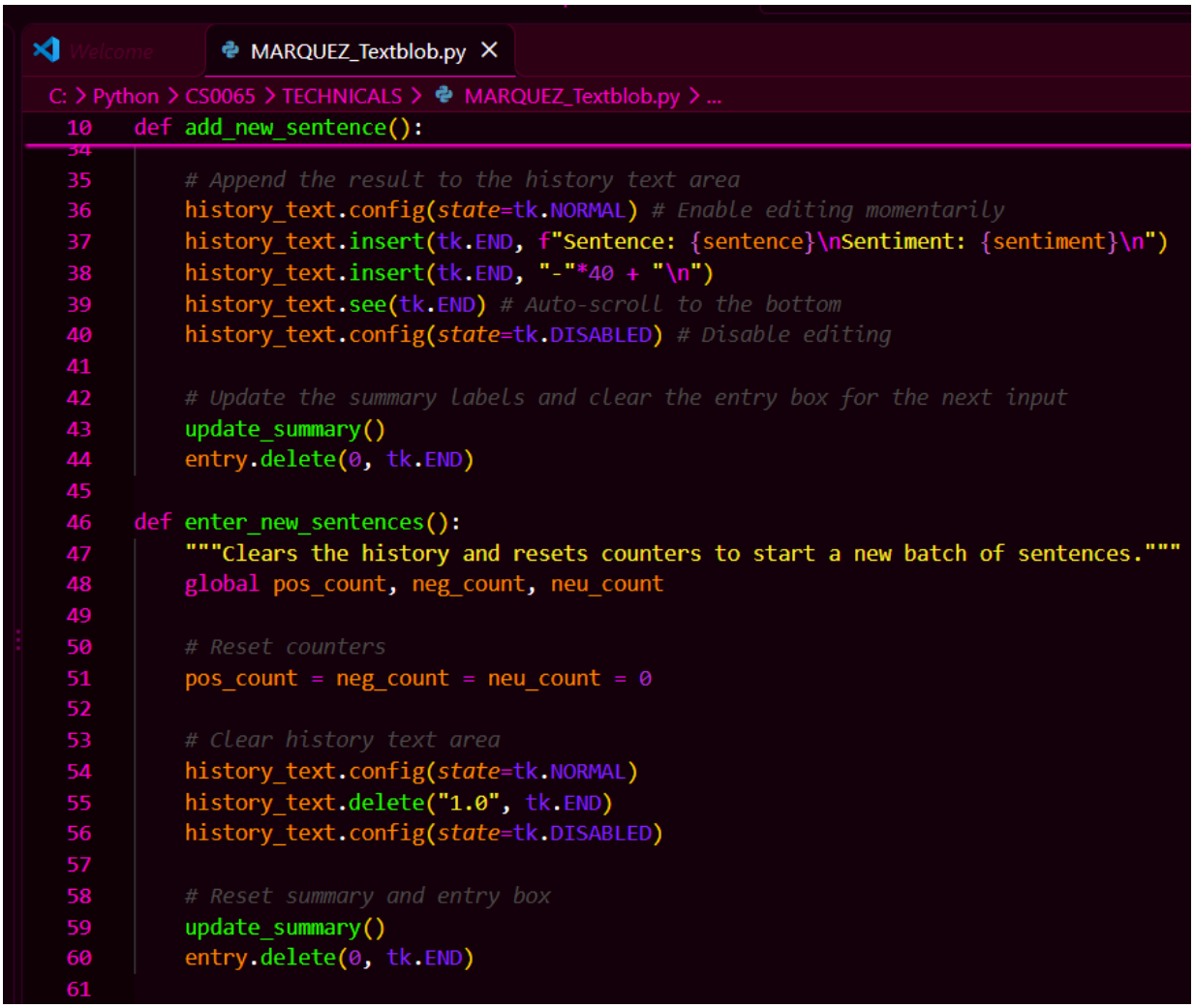
    # Get the sentence from the single-line entry
    sentence = entry.get().strip()

    if not sentence:
        messagebox.showwarning("Input Error", "Please enter a sentence.")
        return

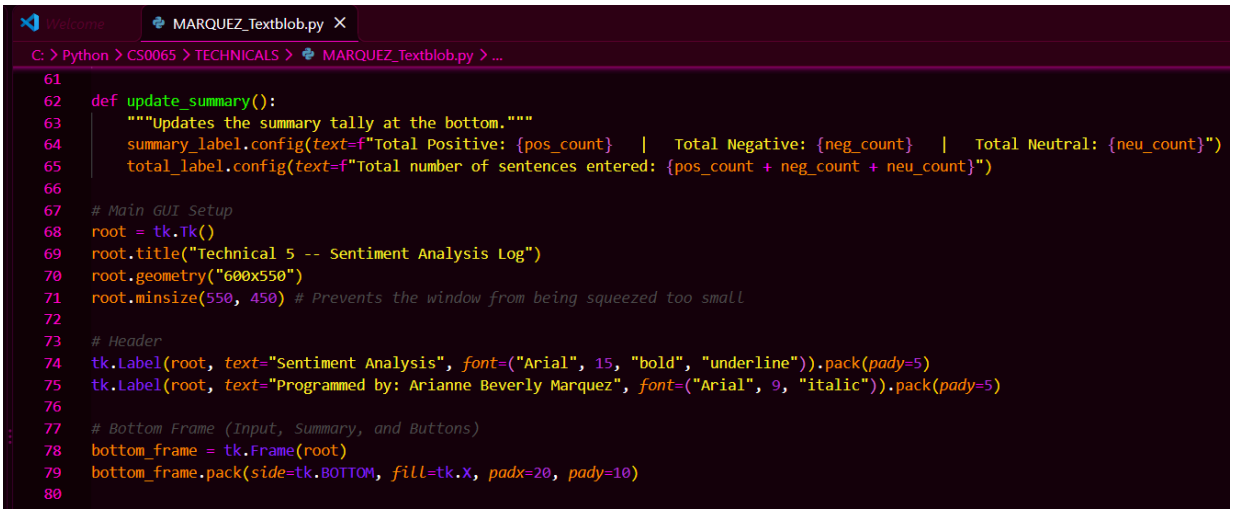
    # TextBlob sentiment analysis
    s = TextBlob(sentence)
    p = s.polarity

    # Make a decision based on the polarity
    if p < 0:
        sentiment = "Negative"
        neg_count += 1
    elif p == 0:
        sentiment = "Neutral"
        neu_count += 1
    elif p > 0 and p <= 1:
        sentiment = "Positive"
        pos_count += 1

```



```
Welcome  MARQUEZ_Textblob.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Textblob.py > ...
10    def add_new_sentence():
35        # Append the result to the history text area
36        history_text.config(state=tk.NORMAL) # Enable editing momentarily
37        history_text.insert(tk.END, f"Sentence: {sentence}\nSentiment: {sentiment}\n")
38        history_text.insert(tk.END, "-"*40 + "\n")
39        history_text.see(tk.END) # Auto-scroll to the bottom
40        history_text.config(state=tk.DISABLED) # Disable editing
41
42        # Update the summary labels and clear the entry box for the next input
43        update_summary()
44        entry.delete(0, tk.END)
45
46    def enter_new_sentences():
47        """Clears the history and resets counters to start a new batch of sentences."""
48        global pos_count, neg_count, neu_count
49
50        # Reset counters
51        pos_count = neg_count = neu_count = 0
52
53        # Clear history text area
54        history_text.config(state=tk.NORMAL)
55        history_text.delete("1.0", tk.END)
56        history_text.config(state=tk.DISABLED)
57
58        # Reset summary and entry box
59        update_summary()
60        entry.delete(0, tk.END)
61
```



```
Welcome  MARQUEZ_Textblob.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Textblob.py > ...
61
62    def update_summary():
63        """Updates the summary tally at the bottom."""
64        summary_label.config(text=f"Total Positive: {pos_count}   |   Total Negative: {neg_count}   |   Total Neutral: {neu_count}")
65        total_label.config(text=f"Total number of sentences entered: {pos_count + neg_count + neu_count}")
66
67    # Main GUI Setup
68    root = tk.Tk()
69    root.title("Technical 5 -- Sentiment Analysis Log")
70    root.geometry("600x550")
71    root.minsize(550, 450) # Prevents the window from being squeezed too small
72
73    # Header
74    tk.Label(root, text="Sentiment Analysis", font=("Arial", 15, "bold", "underline")).pack(pady=5)
75    tk.Label(root, text="Programmed by: Arianne Beverly Marquez", font=("Arial", 9, "italic")).pack(pady=5)
76
77    # Bottom Frame (Input, Summary, and Buttons)
78    bottom_frame = tk.Frame(root)
79    bottom_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=20, pady=10)
80
```

```
MARQUEZ_Textblob.py
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Textblob.py > ...
81   # Summary Labels (Inside bottom frame)
82   summary_label = tk.Label(bottom_frame, text="Total Positive: 0   |   Total Negative: 0   |   Total Neutral: 0", font=("Arial", 10, "bold"))
83   summary_label.pack(pady=5)
84
85   total_label = tk.Label(bottom_frame, text="Total number of sentences entered: 0", font=("Arial", 10, "bold"))
86   total_label.pack(pady=2)
87
```

```
88   # Input Field and Instructions (Inside bottom frame)
89   tk.Label(bottom_frame, text="Type a sentence below:").pack(pady=(10, 0))
90   tk.Label(bottom_frame, text="(You may press Enter button on your keyboard or click the 'Enter Sentence' button below)", font=("Arial", 8, "italic"), fg="#555555").pack
91
```

```
91
92   entry = tk.Entry(bottom_frame)
93   entry.pack(fill=tk.X, pady=5) # makes the entry box stretch horizontally with the window
94
95   # Buttons Frame (Inside bottom frame)
96   btn_frame = tk.Frame(bottom_frame)
97   btn_frame.pack(pady=5)
98
99   add_btn = tk.Button(btn_frame, text="Add New Sentence", bg="lightblue", width=18, command=add_new_sentence)
100  add_btn.grid(row=0, column=0, padx=5)
101
102  enter_btn = tk.Button(btn_frame, text="Enter Sentence", bg="lightgreen", width=18, command=add_new_sentence)
103  enter_btn.grid(row=0, column=1, padx=5)
104
105  clear_btn = tk.Button(btn_frame, text="Enter New Sentence/s (Reset)", bg="#ffb3b3", width=24, command=enter_new_sentences)
106  clear_btn.grid(row=0, column=2, padx=5)
107
```

```
108  #  Middle Frame (History Text Area)
109  history_text = tk.Text(root, state=tk.DISABLED, bg="#f0f0f0")
110  history_text.pack(fill=tk.BOTH, expand=True, padx=20, pady=5)
111
112  # Bind the Enter key
113  root.bind('<Return>', lambda event: add_new_sentence())
114
115  root.mainloop()
```

VII. CHALLENGE

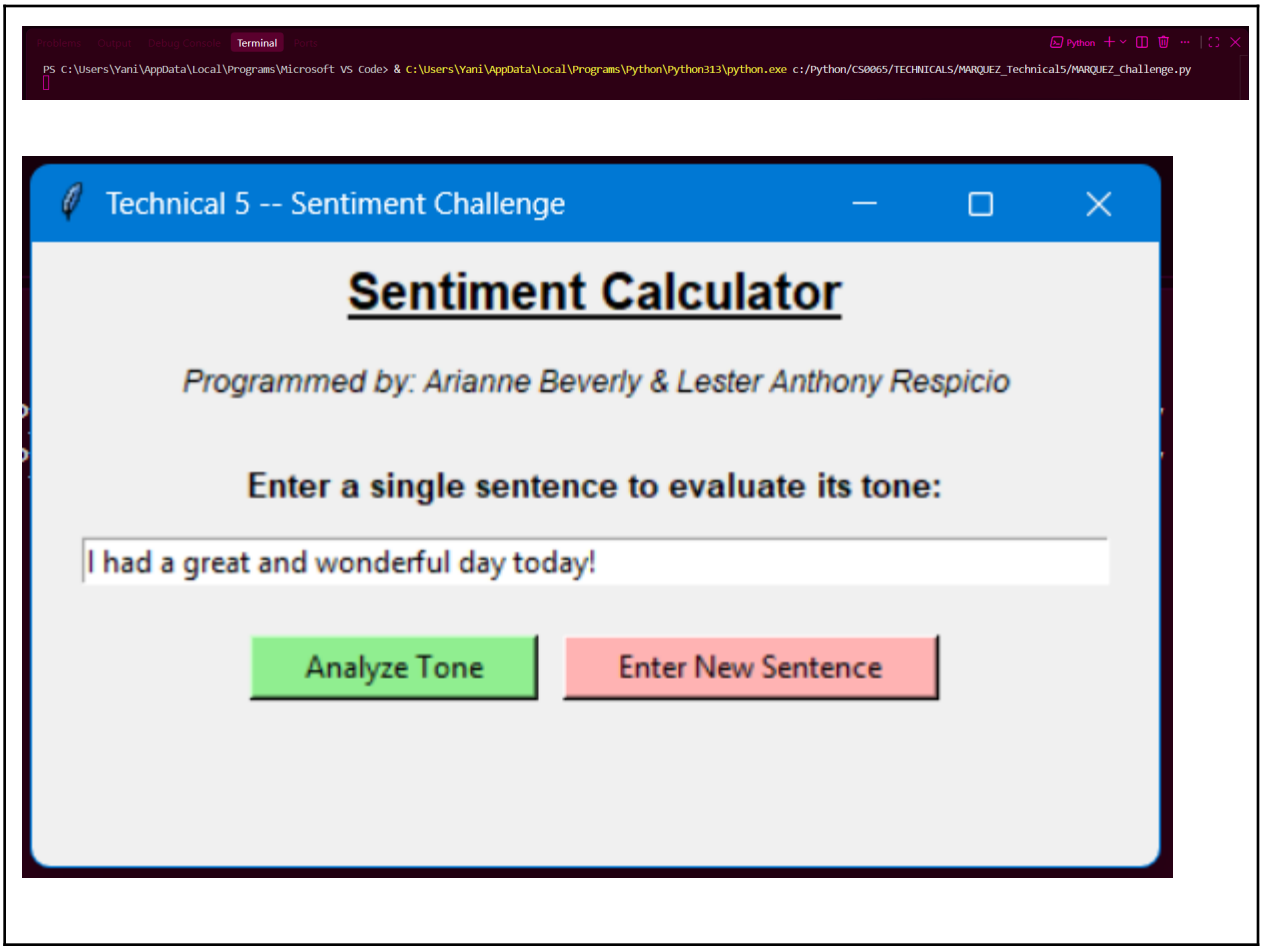
Write a program that will ask the user for a string containing a sentence. The program will then compute the happiness and sadness level of the sentence using the two functions described below. If the happiness level is higher than sadness level, then the tone of the sentence is happy. If the sadness level is higher, then the tone of the sentence is sad. Otherwise, it is neutral.

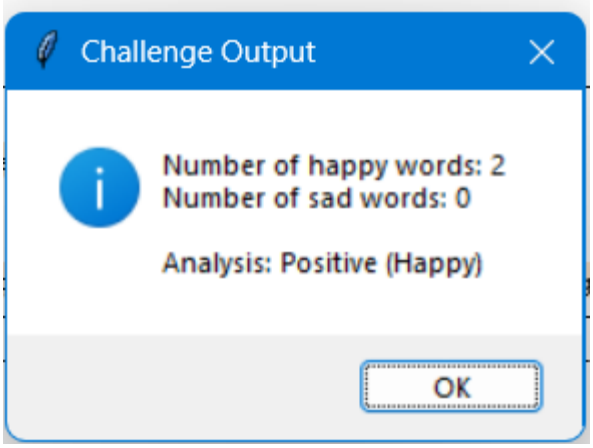
Find and print the tone of the sentence by printing the number of happy words followed by the number of sad words, followed by a simple statement of the analysis (whether positive, negative or neutral).

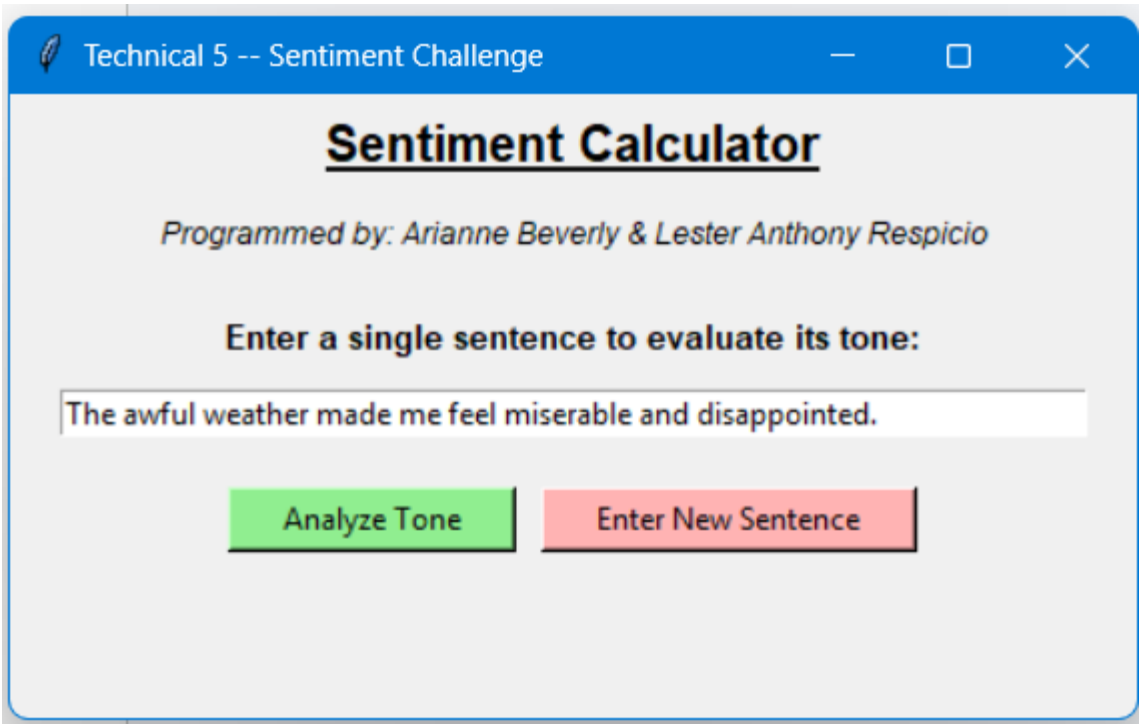
To accomplish this, you will write a function called number_happy(sentence) which returns the number of words in a given string called “sentence” that are happy. To do this, look for a public dataset containing a list of happy words. Find the total number of words in “sentence” that are in the happy words dataset.

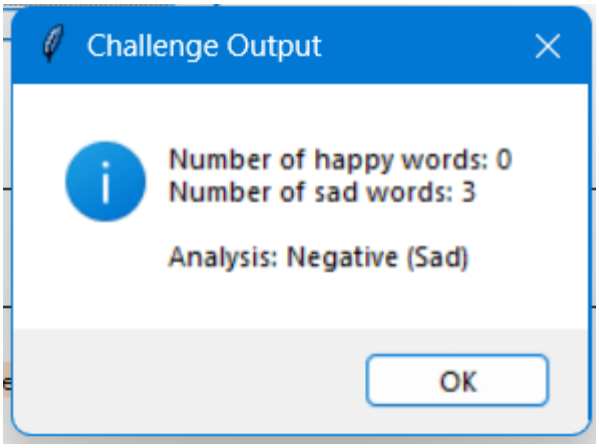
Next, write a second function called number_sad(sentence) that works the same way but instead counts the number of sad words, also from a separate dataset.

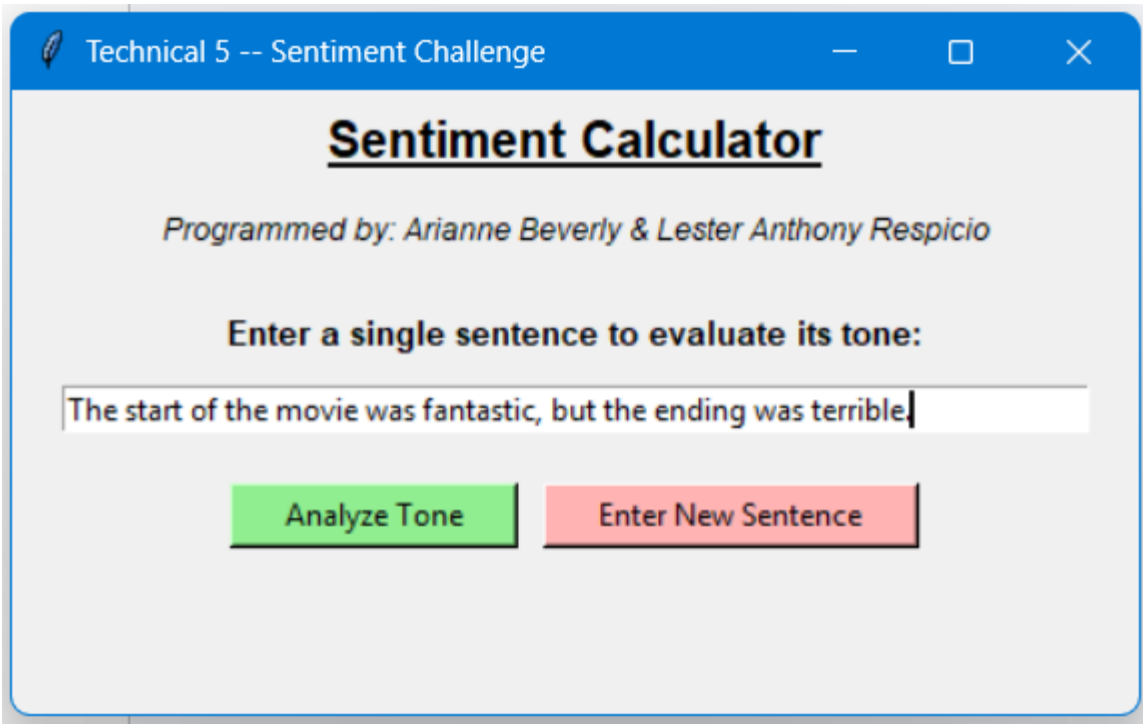
OUTPUT:

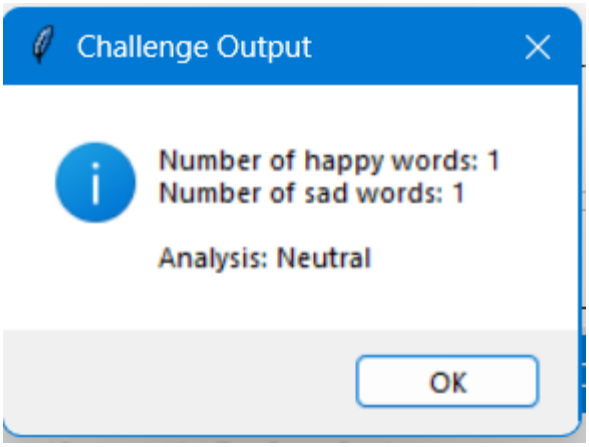



Challenge Output
Number of happy words: 2
Number of sad words: 0
Analysis: Positive (Happy)
OK


Technical 5 -- Sentiment Challenge
Sentiment Calculator
Programmed by: Arianne Beverly & Lester Anthony Respicio
Enter a single sentence to evaluate its tone:
The awful weather made me feel miserable and disappointed.
Analyze Tone
Enter New Sentence


Challenge Output
Number of happy words: 0
Number of sad words: 3
Analysis: Negative (Sad)
OK





CODE:

```
import tkinter as tk
from tkinter import messagebox

# Datasets
happy_dataset = ["happy", "joy", "cheerful", "good", "great",
"excellent", "love", "smile", "fantastic", "wonderful", "delight"]
sad_dataset = ["sad", "sorrow", "depressed", "bad", "terrible",
"awful", "hate", "cry", "miserable", "tragic", "grief",
"disappointed"]
```

```
def number_happy(sentence):
    """Returns the number of words in a given string that are in the
happy dataset."""
    count = 0
    for word in sentence.lower().split():
        clean_word = word.strip(".,!?\"'") # Strip punctuation for
accurate matching
        if clean_word in happy_dataset:
            count += 1
    return count

def number_sad(sentence):
    """Returns the number of words in a given string that are in the
sad dataset."""
    count = 0
    for word in sentence.lower().split():
        clean_word = word.strip(".,!?\"'")
        if clean_word in sad_dataset:
            count += 1
    return count

def evaluate_challenge():
    sentence = entry.get()

    if not sentence.strip():
        messagebox.showwarning("Input Error", "Please enter a
sentence to analyze.")
        return

    happy_count = number_happy(sentence)
    sad_count = number_sad(sentence)

    # Find the tone of the sentence based on the higher level
    if happy_count > sad_count:
        analysis = "Positive (Happy)"
    elif sad_count > happy_count:
        analysis = "Negative (Sad)"
    else:
        analysis = "Neutral"

    # Format and display the output
    result = f"Number of happy words: {happy_count}\n"
    result += f"Number of sad words: {sad_count}\n\n"
    result += f"Analysis: {analysis}"
```

```
    messagebox.showinfo("Challenge Output", result)

def reset_entry():
    """Clears the entry box for a new sentence."""
    entry.delete(0, tk.END)

# Main GUI Setup
root = tk.Tk()
root.title("Technical 5 -- Sentiment Challenge")
root.geometry("450x250")
root.minsize(400, 250) # Prevents the window from becoming too small

# Header Labels
tk.Label(root, text="Sentiment Calculator", font=("Arial", 15,
"bold", "underline")).pack(pady=5)
tk.Label(root, text="Programmed by: Arianne Beverly & Lester Anthony
Respicio", font=("Arial", 9, "italic")).pack(pady=5)

# Content Frame
content_frame = tk.Frame(root)
content_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=5)

tk.Label(content_frame, text="Enter a single sentence to evaluate
its tone:", font=("Arial", 10, "bold")).pack(pady=(10, 5))

# Input Field
entry = tk.Entry(content_frame)
entry.pack(fill=tk.X, pady=5) # allows it to stretch horizontally
with the window)

# Buttons Frame to keep them side-by-side
btn_frame = tk.Frame(content_frame)
btn_frame.pack(pady=15)

# Evaluate Button
btn = tk.Button(btn_frame, text="Analyze Tone", bg="lightgreen",
width=15, command=evaluate_challenge)
btn.grid(row=0, column=0, padx=5)

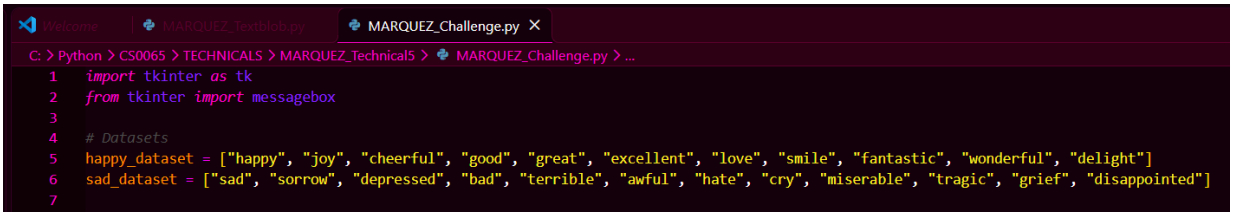
# Reset Button
reset_btn = tk.Button(btn_frame, text="Enter New Sentence",
bg="#ffb3b3", width=20, command=reset_entry)
reset_btn.grid(row=0, column=1, padx=5)

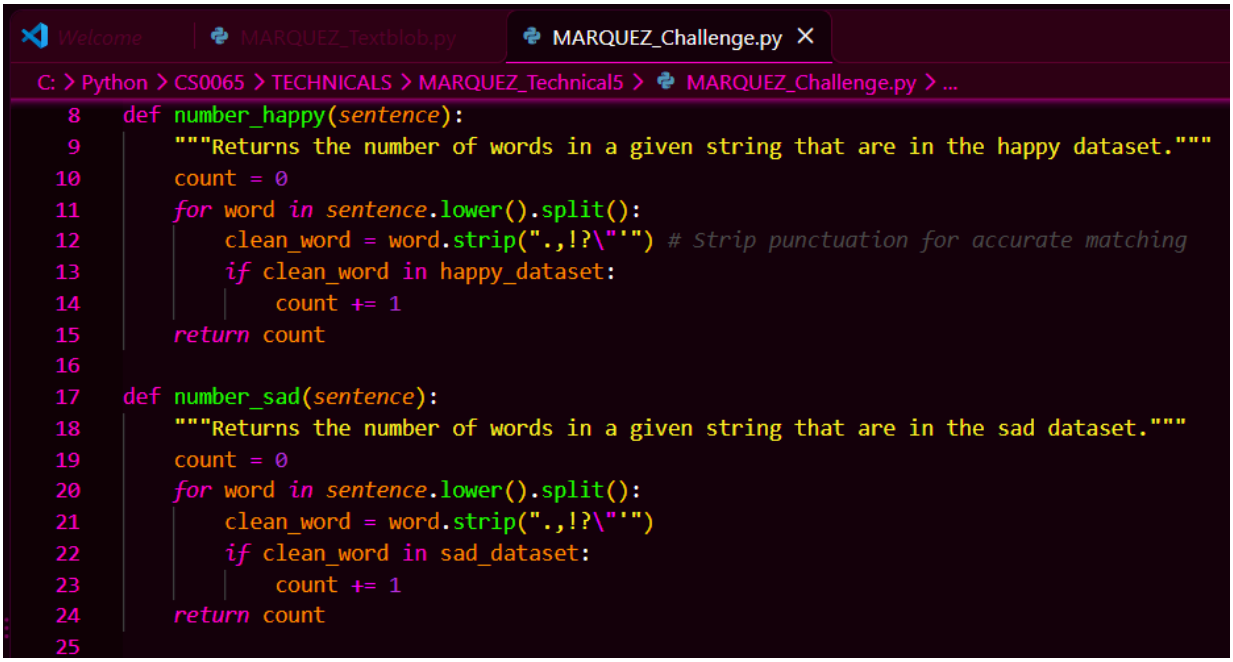
# Bind the Enter key to automatically trigger the analysis
```

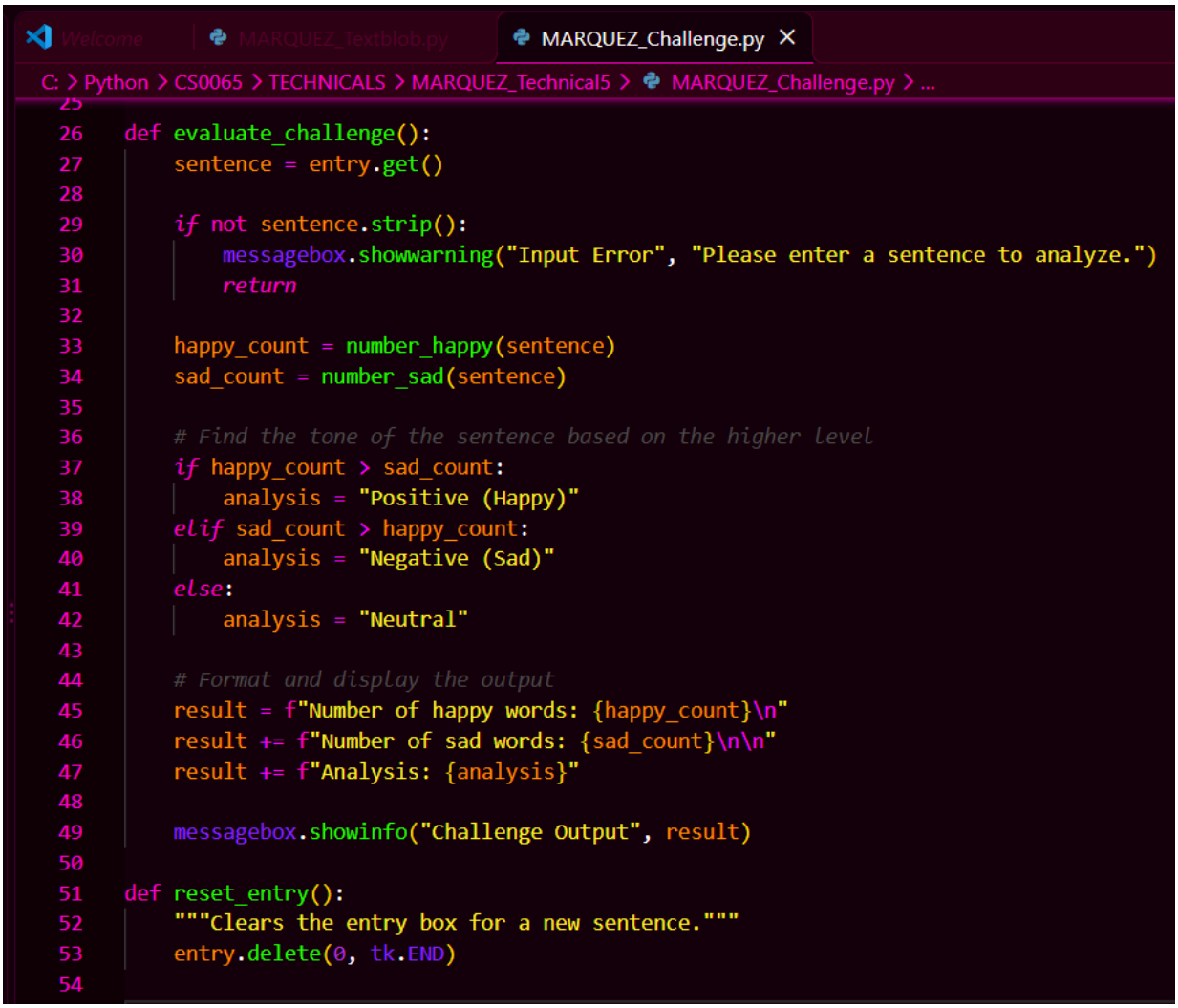
```
root.bind('<Return>', lambda event: evaluate_challenge())

root.mainloop()
```

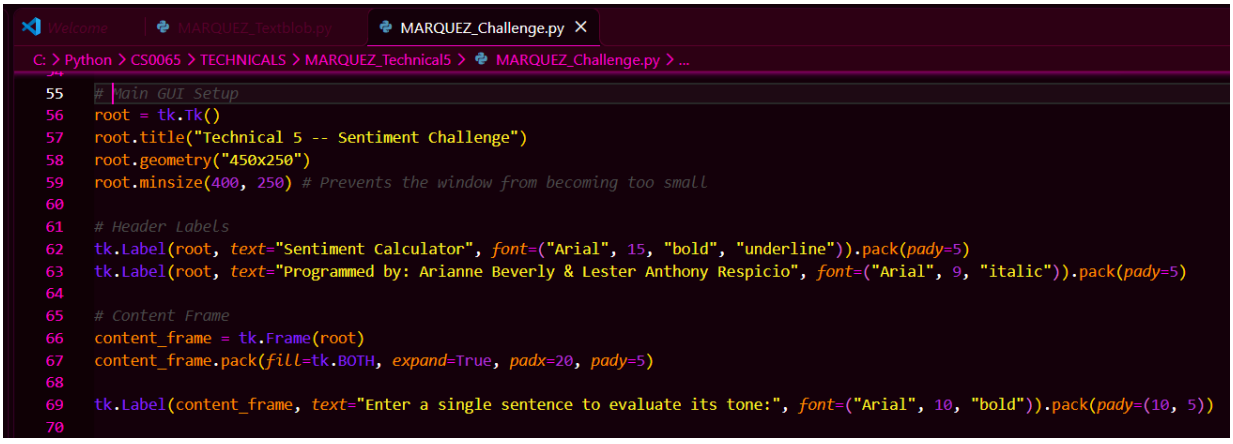
CODE SCREENSHOT:







```
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Technical5 > MARQUEZ_Challenge.py > ...
25
26  def evaluate_challenge():
27      sentence = entry.get()
28
29      if not sentence.strip():
30          messagebox.showwarning("Input Error", "Please enter a sentence to analyze.")
31          return
32
33      happy_count = number_happy(sentence)
34      sad_count = number_sad(sentence)
35
36      # Find the tone of the sentence based on the higher level
37      if happy_count > sad_count:
38          analysis = "Positive (Happy)"
39      elif sad_count > happy_count:
40          analysis = "Negative (Sad)"
41      else:
42          analysis = "Neutral"
43
44      # Format and display the output
45      result = f"Number of happy words: {happy_count}\n"
46      result += f"Number of sad words: {sad_count}\n\n"
47      result += f"Analysis: {analysis}"
48
49      messagebox.showinfo("Challenge Output", result)
50
51  def reset_entry():
52      """Clears the entry box for a new sentence."""
53      entry.delete(0, tk.END)
54
```



```
C: > Python > CS0065 > TECHNICALS > MARQUEZ_Technical5 > MARQUEZ_Challenge.py > ...
55  # Main GUI Setup
56  root = tk.Tk()
57  root.title("Technical 5 -- Sentiment Challenge")
58  root.geometry("450x250")
59  root.minsize(400, 250) # Prevents the window from becoming too small
60
61  # Header Labels
62  tk.Label(root, text="Sentiment Calculator", font=("Arial", 15, "bold", "underline")).pack(pady=5)
63  tk.Label(root, text="Programmed by: Arianne Beverly & Lester Anthony Respicio", font=("Arial", 9, "italic")).pack(pady=5)
64
65  # Content Frame
66  content_frame = tk.Frame(root)
67  content_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=5)
68
69  tk.Label(content_frame, text="Enter a single sentence to evaluate its tone:", font=("Arial", 10, "bold")).pack(pady=(10, 5))
70
```

```
# Input Field
entry = tk.Entry(content_frame)
entry.pack(fill=tk.X, pady=5) # allows it to stretch horizontally with the window)

# Buttons Frame to keep them side-by-side
btn_frame = tk.Frame(content_frame)
btn_frame.pack(pady=15)

# Evaluate Button
btn = tk.Button(btn_frame, text="Analyze Tone", bg="lightgreen", width=15, command=evaluate_challenge)
btn.grid(row=0, column=0, padx=5)

# Reset Button
reset_btn = tk.Button(btn_frame, text="Enter New Sentence", bg="#ffb3b3", width=20, command=reset_entry)
reset_btn.grid(row=0, column=1, padx=5)

# Bind the Enter key to automatically trigger the analysis
root.bind('<Return>', lambda event: evaluate_challenge())

root.mainloop()
```

VIII. REFERENCES

- Artasanchez A. Joshi P.(2020). Artificial Intelligence with Python: You Complete Guide to Building Intelligent apps using Python 3.x 2nd Edition. Packt Publishing
- Attaway, Stormy (2013). MATLAB ®. A Practical Introduction to Programming and Problem Solving Third Edition. Elsevier, Inc.

Online Sources:

- https://www.cuelogic.com/blog/role-of-python-in-artificial-intelligencehttps://monkeylearn.com/sentiment analysis
- https://towardsdatascience.com/a-beginners-guide-to-sentiment-analysis-in-python-95e354ea84f6 • https://research.google.com/colaboratory
- https://www.youtube.com/watch?v=RLYoEyIHL6A